学校代码：11059

学　　号：_____

合肥大學

HEFEI UNIVERSITY

毕业设计（论文）

BACHELOR DISSERTATION

论文题目：算法演示教学系统的设计与实现

学位类别：工学学士

学科专业：_____

作者姓名：_____

导师姓名：_____

校外辅助指导教师:_____无

完成时间：2026 年__月__日

合肥大学学位论文原创性声明

郑重声明：本人所呈交的学位论文

算法演示教学系统的设计与实现

是在导师的指导下，独立进行研究取得的成果。除文中已经注明引用的内容外，本论文不含其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果，并承诺因本声明而产生的的法律结果由本人承担。

学位论文作者：

日期： 年 月 日

合肥大学学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权合肥大学将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本学位论文属于

保 密□，在__年解密后适用本授权书。

不保密 ☑。

学位论文作者：　　指导教师：

日期：　年　月　日　　日期：　年　月

算法演示教学系统的设计与实现

中 文 摘 要

算法课程是计算机专业核心基础课程，但在实际教学中长期存在“知识点难讲透、学生难跟上、练习反馈慢”的共性问题。传统课堂通常依赖静态板书、伪代码推导和课后习题，虽然能够覆盖知识框架，但对算法运行过程中“变量如何变化、数据结构如何演进、关键分支何时触发”的解释能力有限。特别是在图算法和动态规划等主题中，学生往往能背诵结论，却难以建立对执行过程的动态认知，导致学习效果不稳定。针对这一问题，本文结合当前项目工程实现，设计并落地一个面向算法课程教学场景的算法演示教学系统，核心目标是将“看算法、练算法、评算法、追踪学习过程”整合为统一闭环。

系统采用前后端分离架构，前端以 Vue3 + Pinia + D3 构建交互式可视化界面，后端以 Node.js + Express 提供算法执行、鉴权认证、学习进度管理和练习判分接口，数据持久化采用 SQLite。系统支持 10 种典型算法：冒泡排序、快速排序、选择排序、插入排序、二分查找、深度优先搜索、广度优先搜索、Dijkstra、二叉搜索树插入和 0/1 背包。后端将算法执行抽象为“快照序列（AlgorithmSnapshot[]）”，每一步快照包含数据状态、伪代码高亮行、当前节点和变量字典。前端根据快照统一驱动可视化渲染，实现数组柱图、图结构、树结构、动态规划矩阵四类视图，并提供播放、暂停、逐步前进/后退和速度调节控制。

在学习闭环方面，系统实现了“选择题 + 代码填空题”的双轨练习机制。填空题判分模块优先调用 DeepSeek 语义评测接口，返回是否正确、分数和讲评；当外部服务不可用时自动退化为本地规则判分，保证核心功能不中断。用户登录后可持续记录学习进度（完成状态、得分、最后访问时间），支持阶段性复盘。

本文在真实代码基础上完成了功能测试、算法正确性测试和接口联调测试。测试结果表明：系统核心链路稳定、算法输出正确、可视化过程连续可解释，能够有效支撑“课堂演示 + 课后自学 + 形成性反馈”三类教学场景。本文工作为算法课程数字化教学提供了可部署、可扩展、可维护的工程方案，并为后续教师端管理、学习画像和个性化推荐奠定了实现基础。

关键词：算法可视化；教学系统；交互练习；学习进度追踪；前后端分离

Design and Implementation of an Algorithm Demonstration Teaching System

ABSTRACT

Algorithm education is a foundational part of computer science curricula, yet conventional teaching approaches still face practical limitations in process explanation, individualized pacing, and timely feedback. In many classrooms, algorithm learning mainly depends on static pseudocode explanation

and after-class exercises. Such methods can deliver concepts but often fail to show how state transitions actually happen during execution. As a result, students may remember formulas or complexity results but struggle to understand why each step is necessary. This issue is especially significant in graph algorithms and dynamic programming.

To address these problems, this thesis presents a full-stack implementation of an Algorithm Demonstration Teaching System. The system is designed around an integrated loop of demonstration, practice, evaluation, and learning tracking. A front-end/back-end separated architecture is adopted. The front end is built with Vue3, Pinia, and D3 for interactive visualization and state-driven rendering. The back end is implemented with Node.js and Express, providing algorithm execution APIs, authentication services, progress persistence, and practice grading services. SQLite is used for lightweight local persistence.

The platform currently supports ten representative algorithms, including Bubble Sort, Quick Sort, Selection Sort, Insertion Sort, Binary Search, DFS, BFS, Dijkstra, BST Insertion, and 0/1 Knapsack. A unified snapshot model (AlgorithmSnapshot[]) is introduced: each step stores data-structure state, highlighted pseudocode lines, active node information, and key variables. This model enables synchronized animation, code highlighting, and variable tracing in the client player.

To improve learning effectiveness, the system integrates multiple-choice and code fill-in practice. Fill-in grading first uses DeepSeek semantic evaluation and returns correctness, score, and textual feedback. If the external model service is unavailable, a local fallback evaluator is triggered automatically, ensuring service reliability. Authenticated users can continuously store progress records, including completion state, score, and last-visit timestamp.

Comprehensive validation was conducted on real project code, including functional tests, algorithm correctness tests, and API integration tests. Results show that the platform is stable, outputs are correct, and execution steps are explainable. The system effectively supports classroom demonstration and self-paced learning while providing actionable feedback. This work offers a practical engineering reference for digital algorithm education and provides a scalable foundation for future teacher-oriented analytics and adaptive learning paths.

KEY WORDS: algorithm visualization; teaching system; interactive practice; learning progress tracking; front-end/back-end separation

目　　录

中　文　摘　要.....4
ABSTRACT.....4
目　　录.....6

第一章 绪论

第二章 关键技术与理论基础

第三章 需求分析与可行性分析

第四章 系统设计

第五章 系统实现

第六章 系统测试

结论

参考文献

致谢

附录

第一章 绪论

1.1 研究背景

在算法课程教学实践中，学生理解障碍通常并非来源于“看不懂语法”，而是来源于“看不见过程”。以冒泡排序为例，学生往往知道时间复杂度是 O(n^2)，但不知道每一轮比较具体发生了什么；以 Dijkstra 为例，学生能记住“每次选最小距离节点”，却不清楚优先队列和松弛操作之间的协作顺序。传统的讲授方式很难把这种动态过程完整呈现给全体学生，尤其在课堂时间有限的条件下，教师难以对每位学生进行个性化纠偏。

另一方面，算法学习具有明显的“步骤依赖”特征：前一步理解错误会直接影响后续推导。例如，二分查找中 `left/right` 边界更新如果理解不清，学生会在后续所有变体题中反复出错；动态规划中状态定义不清，会导致状态转移式错误扩散。因此，算法教学不仅需要结论正确，更需要过程可解释、反馈及时、练习可追踪。

本课题基于现有项目代码开展，目标并非仅搭建“动画展示网页”，而是构建可落地的教学系统：既能用于课堂演示，也能支持课后自主训练和学习数据沉淀。系统强调工程可维护性，采用前后端分离与统一快照模型，使新增算法、扩展题型和接入教师端成为可持续演进的能力。

1.2 研究目的与意义

1.2.1 研究目的

本文的研究目标分为教学目标和工程目标两层：

1. 教学目标：把抽象算法过程转化为可见、可控、可回放的学习对象，降低学生理解门槛。
2. 训练目标：提供与算法过程紧密耦合的练习机制，而非孤立题库，形成“演示后即练习”的闭环。
3. 反馈目标：在练习阶段输出解释性反馈，而不是仅给出对错，提高学生纠错效率。
4. 追踪目标：记录学习行为与结果，支撑阶段性复盘和后续教学分析。
5. 工程目标：构建可维护、可扩展、可部署的系统架构，支持长期迭代。

1.2.2 研究意义

从教学实践看，本系统将“伪代码、变量、结构状态、语言解释”四种信息同步呈现，能够显著提升课堂可讲性和学生自学效率。从工程实践看，系统验证了“快照驱动可视化”模型在教学平台中的可行性，兼顾了可解释性与可扩展性。从研究价值看，本文将算法教学需求落地到具体实现细节，避免“只讲理念不讲系统”，为同类教学平台提供了可复用方案。

1.3 国内外研究现状

国外在算法教育工具方面较早引入可视化理念，重点解决“过程理解”与“认知负荷”问题，形成了动画演示、交互式步骤控制、即时评测等技术路线。代表性工作普遍强调数据驱动渲染和细粒度反馈，近年又进一步引入学习分析和智能推荐能力。

国内相关研究与产品近年发展较快，多聚焦在线判题、课程平台整合和课堂数字化支撑。但从教学使用角度看，仍存在三类典型缺口：第一，演示系统与练习系统割裂，学生看完演示无法立刻在同一语境下训练；第二，反馈解释不足，很多平台只给出标准答案，不给出思路讲评；第三，学习进度数据孤立，难以形成连续的学习轨迹和改进建议。

本文在这些问题上给出工程化应对：使用统一算法元数据与快照模型连接“演示-练习-反馈-进度”四个环节；通过大模型判分与本地规则兜底结合，提升反馈质量与可用性；通过 `progress` 表持久化关键学习状态，为后续学习分析留出接口。

1.4 研究内容与方法

本文采用“需求分析—架构设计—模块实现—测试验证—总结优化”的路径推进。

1. 需求分析：从课堂演示、自主学习、作业训练三个场景抽取功能需求。
2. 架构设计：定义前后端边界、数据模型、接口契约和安全机制。
3. 模块实现：实现算法引擎、可视化渲染、练习判分、用户认证和进度管理。
4. 测试验证：覆盖接口可用性、算法正确性、功能完整性与可解释性。
5. 经验总结：归纳系统优势、性能瓶颈和可拓展方向。

1.5 论文组织结构

全文共六章：第一章说明研究背景与目标；第二章介绍关键技术；第三章分析需求与可行性；第四章给出系统设计；第五章展开实现细节；第六章给出测试过程与结果，最后总结全文并提出后续工作。

第二章 关键技术与理论基础

2.1 前后端分离架构

系统采用典型 B/S 结构，并使用前后端分离模式组织项目代码。前端负责交互与可视化，后端负责业务逻辑和数据持久化。该模式具备三个直接优势：

1. 前后端并行开发，降低改动耦合。

2. API 契约清晰，方便后续接入移动端或小程序端。

3. 后端可独立测试和部署，保障核心算法服务稳定性。

2.2 前端技术栈

2.2.1 Vue3 组件化

`App.vue` 作为主容器，组织侧边算法列表、主演示区和辅助信息区。核心组件包括：`DataStructureVisualizer.vue` 、 `AlgorithmPlayer.vue` 、 `CodeHighlighter.vue` 、 `InteractiveExercise.vue`、`TheoryPanel.vue` 与 `ProgressCard.vue`。组件边界清晰，便于后续替换和扩展。

2.2.2 Pinia 状态管理

`stores/algorithm.ts` 统一管理算法元数据、快照列表、当前步骤、播放速度等状态。播放器操作（next/prev/play/pause/reset）直接驱动全局状态变化，确保各组件同步刷新。`stores/auth.ts` 管理用户 token 与登录态，封装注册登录退出逻辑。

2.2.3 D3 可视化渲染

`DataStructureVisualizer.vue` 根据算法类型动态选择渲染器：

1. array：柱状图表现排序与查找过程。

2. graph：力导向布局展示访问顺序。

3. tree：层次布局显示 BST 插入路径。

4. dp：网格矩阵展示状态转移填表过程。

D3 的数据绑定机制与快照模型天然匹配，能够稳定表达“步骤—状态”映射关系。

2.3 后端技术栈

2.3.1 Express 路由层

后端入口为 `server/src/index.ts`，注册三组核心路由：

1. `/api/auth`：注册和登录。
2. `/api/algorithms`：算法元数据、执行、填空判分。
3. `/api/progress`：进度读取与写入。

此外提供 `/api/health` 健康检查接口，便于调试和部署巡检。

在实现层面，`server/src/index.ts` 统一挂载了四类入口：`/api/auth`、`/api/algorithms`、`/api/progress` 与 `/api/health`。其中 `/api/health` 返回固定 JSON（status/message），主要用于启动后自检和部署巡检。

`/api/algorithms` 路由不仅提供算法元数据查询（GET），还通过 `POST /api/algorithms/:id/execute` 将前端输入映射到具体引擎。实际请求体字段按算法类别区分：排序使用 `data`，二分查找增加 `target`，图算法使用 `nodes/edges/startNode`，背包算法使用 `weights/values/capacity`。

路由层对异常做了分级返回：参数缺失通常返回 400，未找到算法或题目返回 404，鉴权失败返回 401，内部执行异常返回 500。这样前端可以基于状态码给出更准确的用户提示，而不是统一的“请求失败”。

2.3.2 算法引擎层

后端将算法逻辑拆分为多个引擎类：`SortingEngine`、`SearchingEngine`、`GraphEngine`、`TreeEngine`、`DPEngine`。每个引擎继承统一抽象 `AlgorithmEngine`，对外输出统一快照结构，降低可视化适配成本。

2.3.3 安全机制

认证采用 JWT，令牌默认 7 天有效。密码通过 bcrypt 哈希存储。进度接口使用 `authMiddleware` 强制校验 Bearer Token，避免匿名写入。

账号安全采用“哈希存储 + 令牌鉴权”双层方案。注册接口在 `routes/auth.ts` 中调用 `bcrypt.hash(password, 10)` 生成 `passwordHash`，数据库中不保存明文密码；登录时使用 `bcrypt.compare` 校验，避免可逆密码泄露风险。

登录成功后服务端通过 `generateToken` 签发 JWT，载荷为 `userId`，默认有效期 `7d`。密钥来自环境变量 `JWT_SECRET`，未配置时回退到默认值，便于开发联调，但生产部署应强制改为自定义高强度密钥。

受保护接口（如进度读写）统一接入 `authMiddleware`：校验 `Authorization: Bearer <token>` 头，`verifyToken` 失败即返回 401，不允许进入业务层。这样鉴权逻辑从业务代码中抽离，保证了安全策略的一致性。

2.4 数据存储

系统采用 `better-sqlite3` 操作本地数据库 `database.sqlite`。数据库初始化脚本位于 `server/src/db/index.ts`，自动建表 `users` 与 `progress`。SQLite 的轻量特性适合教学场景，部署简洁且维护成本低。

系统当前采用 `better-sqlite3` 直接访问本地文件库，数据库路径由 `path.join(process.cwd(), 'database.sqlite')` 计算得到。该方案在教学场景下具备部署简单、依赖少、迁移成本低的优势。

`users` 表结构为 `id/username/passwordHash/createdAt`，其中 `id` 为主键，`username` 设置唯一约束，用于防止重复注册。`progress` 表以 `(userId, algorithmId)` 作为联合主键，字段包含 `completed/score/lastVisit`，用于记录某个用户在某个算法上的阶段状态。

进度写入采用“先查后更”的 Upsert 逻辑：先查询记录是否存在，存在则 `UPDATE`，不存在则 `INSERT`。另外 `completed` 在数据库层以 `0/1` 存储，在接口输出时转换为布尔值，保证前端状态判断更直接。

2.5 练习判分机制

填空题判分流程：

1. 根据算法 ID 组装题目元数据（题干、模板、可接受答案）。
2. 若配置 `DEEPSEEK_API_KEY`，调用 DeepSeek 语义评估接口返回 JSON 判分结果。
3. 若远程接口异常或未配置密钥，回退到本地标准化字符串比对。
4. 最终统一返回 `isCorrect/score/feedback/expectedAnswer/source`。

该设计兼顾了评估质量与服务可用性。

练习题生成基于`shared/practice.ts`的统一规则：每个算法默认生成 2 道选择题（概念题 + 计算题）和 1 道代码填空题。题目和选项并非纯静态文本，而是结合算法类别和当前快照变量动态组织。

填空题判分优先调用`gradeFillBlankWithDeepSeek`。请求固定使用`temperature=0`，并要求模型只返回 JSON 字段`isCorrect/score/feedback`，降低模型输出漂移对系统解析的影响。

为保证服务可用性，判分链路内置兜底：当`DEEPSEEK_API_KEY`缺失、远端接口异常或返回 JSON 不可解析时，系统自动走本地规则判分（`normalizePracticeAnswer + acceptableAnswers`），并在响应中标记`source=fallback`。

第三章 需求分析与可行性分析

3.1 角色分析与场景分析

系统当前主要面向学生，但可兼容教师课堂使用。角色划分如下：

1. 未登录访客：可浏览算法与演示。
2. 登录用户：可进行练习、判分与进度记录。
3. 教师角色（扩展位）：可用于投屏讲解和课堂引导。

典型场景包括：

1. 课堂演示：教师选定算法并逐步讲解关键步骤。
2. 课后复习：学生在播放器中反复观察状态变化。
3. 针对性训练：学生在演示后立即完成选择题和填空题。
4. 阶段追踪：系统记录学习得分和完成状态。

3.2 功能需求

系统功能需求可分为五组：

1. 算法管理：返回算法名称、复杂度、伪代码和分类。

2. 执行演示：接收输入后输出快照序列并可回放。

3. 练习判分：支持题目交互与讲评反馈。

4. 用户认证：注册、登录、退出、鉴权。

5. 进度追踪：写入并读取用户学习状态。

3.3 非功能需求

非功能需求用于约束系统在真实教学环境中的可持续运行质量，而不仅是功能清单是否齐全。本项目从可用性、可靠性、可维护性、可扩展性、安全性五个方面给出可落地目标。

1. 可用性：系统应支持“开机即用”的课堂节奏。前端以单页方式集成演示、练习、理论三类能力，后端提供 `/api/health` 作为快速可用性检查点，降低课前准备成本。

2. 可靠性：系统应避免单点依赖导致教学中断。智能判分采用“DeepSeek 优先 + 本地规则兜底”，当外部模型异常时仍返回完整判分结构，保证练习链路可继续执行。

3. 可维护性：系统应具备清晰模块边界。后端按 `routes/engines/services/db` 分层，前端以 `stores` 作为状态中枢；`shared/types.ts` 统一跨端数据结构，减少字段不一致造成的联调问题。

4. 可扩展性：系统应支持低成本新增算法与教学模块。新增算法只需补充元数据并映射执行引擎，即可复用快照播放器；学习记录以“用户-算法”为粒度，为后续扩展课程与班级维度预留空间。

5. 安全性：系统应满足基础账号与数据安全要求。密码仅以 bcrypt 哈希保存，受保护接口必须通过 JWT Bearer 认证，中间件校验后再注入 `userId`，避免前端伪造身份参数。

以上指标构成系统质量基线：在网络波动、外部服务异常与高频交互条件下，系统仍应保持教学流程连续、数据语义一致和接口行为可预期。

3.4 可行性分析

3.4.1 技术可行性

项目所用技术均成熟稳定，并且已在当前代码中完成初步落地。前后端框架、数据库和鉴权方案均有充分文档与社区支持，开发风险低。

从技术成熟度看，系统核心依赖均为稳定版本：前端基于 Vue 3 + Vite + Pinia + D3，后端基于 Express 5 + TypeScript + better-sqlite3。各组件均有明确文档与长期社区维护，学习成本和二次开发风险可控。

从实现耦合度看，项目通过 `shared/types.ts` 统一了前后端数据契约（如 `AlgorithmSnapshot`、`UserProgress`、`FillBlankEvaluation`），减少了“前后端字段不一致”导致的联调问题。这也是本项目能够稳定扩充算法数量的关键前提。

从演进空间看，当前 BST 模块的 `buildTree` 仍是简化实现（以首节点构建根），后续可以扩展为完整树重建逻辑。该问题不会影响当前演示主流程，但在复杂树用例下需要补全。

3.4.2 经济可行性

系统全部采用开源组件，部署无需商业授权。SQLite 本地化部署降低了运维成本，适用于实验课程与毕业设计资源条件。

系统采用开源技术栈，开发与部署阶段不依赖商业授权组件；数据库使用 SQLite 单文件存储，无需额外采购数据库服务或维护独立数据库实例。

在教学场景下，单机部署即可完成课程演示与练习闭环，硬件成本主要来自常规教学电脑。若后续扩容，可平滑迁移到 MySQL/PostgreSQL，不会造成前期投入浪费。

3.4.3 运行可行性

运行环境依赖仅为 Node.js 与 npm。项目附带脚本可自动安装依赖并启动服务，适配教学机房和个人电脑环境。

项目提供了脚本化运维入口：`setup.ps1` 负责环境检查与依赖安装，`start.ps1` 负责启动后端（3000）与前端（5173），`stop.ps1` 负责统一停止服务。相较手动多终端启动，脚本方式更适合课程现场快速恢复。

系统具备可观测的运行检查点：后端 `GET /api/health` 可用于确认 API 服务状态；前端若检测到判分服务不可用，会自动切换到本地判分逻辑，避免单一外部依赖导致课堂中断。

第四章 系统设计

4.1 总体架构设计

系统总体架构如图 4.1 所示，分为客户端层、API 层、算法引擎层和数据安全层。

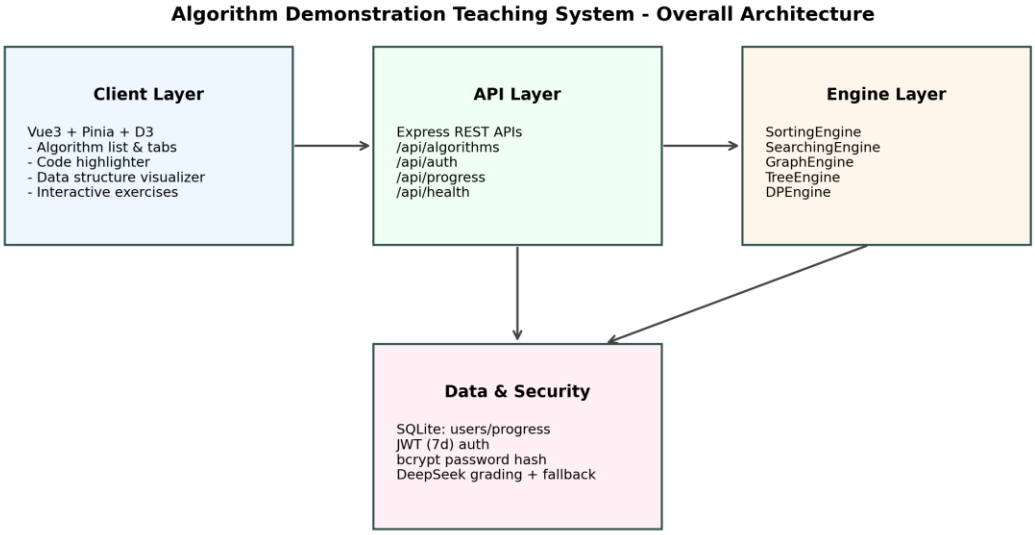

图 4.1 系统总体架构图

在该架构下，所有算法执行结果都被标准化为快照数据，前端无需感知具体算法实现细节，只需消费统一结构并渲染。这种“后端统一建模、前端统一驱动”的模式，是系统可扩展性的核心。

4.2 模块结构设计

4.2.1 后端模块结构

后端模块结构如图 4.2 所示。

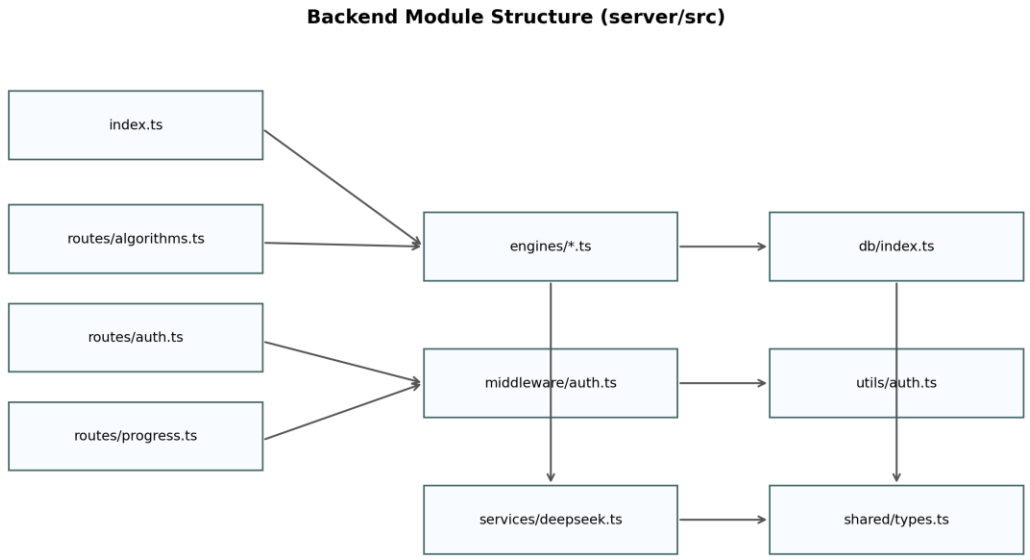

图 4.2 后端模块结构图

路由层负责 HTTP 接口，算法引擎层负责计算逻辑，数据库层负责持久化，服务层封装外部判分能力。各层职责明确，便于定位问题和独立测试。

后端模块按“路由层—引擎层—服务层—存储层”组织：`routes/*` 负责 HTTP 协议与参数校验，`engines/*` 负责算法执行与快照生成，`services/deepseek.ts` 负责外部模型调用，`db/index.ts` 负责持久化。模块边界清晰，便于定位故障来源。

在执行链路上，`routes/algorithms.ts` 通过 `id + type` 映射到具体引擎（如 `bubbleSort`、`dijkstra`、`knapsack`），引擎统一返回 `AlgorithmSnapshot[]`。该抽象使前端不需要感知算法内部差异，降低了跨模块改动成本。

进度写入和用户认证模块与算法引擎解耦，保证了即便后续新增算法，也不会影响鉴权与学习记录机制。

4.2.2 前端模块结构

前端模块结构如图 4.3 所示。

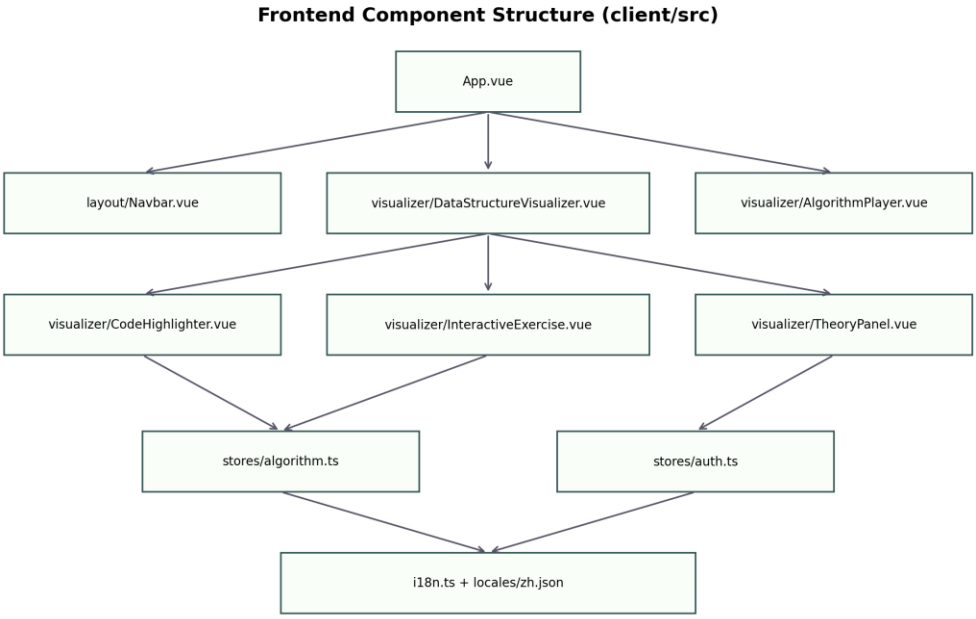

图 4.3 前端组件结构图

其中 `App.vue` 为编排中心，`stores` 为状态中心，`visualizer` 目录为核心交互中心。组件之间通过 Pinia 状态通信，避免层层传参导致的维护复杂度上升。

前端以 `App.vue` 为编排中心，将页面拆分为可视化、练习、理论三个标签页，并通过 `activeTab` 做状态切换。算法切换时统一调用 `selectAlgorithm`，按算法类别构造初始输入并触发后端执行。

`stores/algorithm.ts` 是核心状态容器，维护 `snapshots/currentStepIndex/isPlaying/playbackSpeed` 等关键状态，并提供 `runAlgorithm/nextStep/prevStep/startPlayback/setSpeed` 等动作。播放速度通过滑杆在 50~1000ms 步进调整。

组件层围绕 `currentSnapshot` 消费状态：`DataStructureVisualizer` 负责结构渲染，`CodeHighlighter` 负责伪代码高亮，`ProgressCard` 负责学习进度展示，`InteractiveExercise` 负责题目交互与判分反馈，形成了低耦合协同。

4.3 数据库设计

数据库采用 SQLite（better-sqlite3）进行本地持久化，数据库文件默认位于 `database.sqlite`，在教学机房环境下无需额外安装数据库服务。

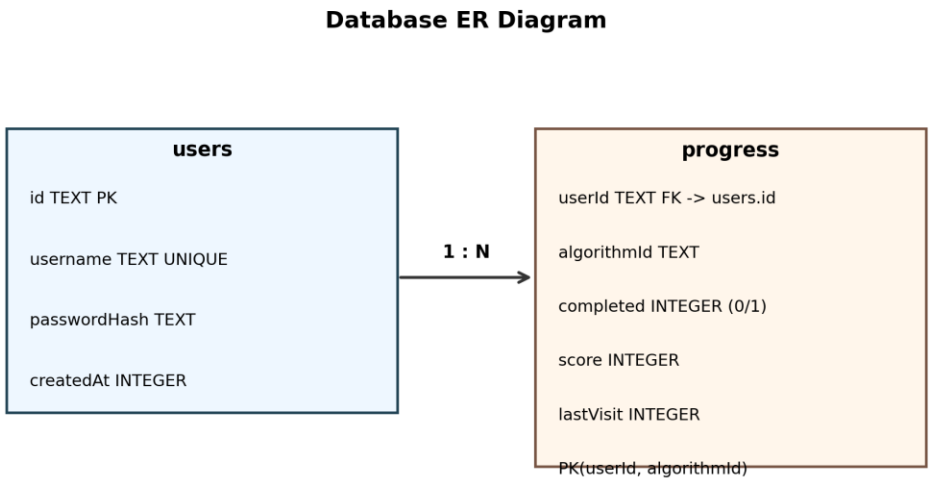

图 4.4 数据库 E-R 图

数据库 E-R 关系如图 4.4 所示。当前模型由 `users` 与 `progress` 两张核心表组成，分别承担身份管理与学习轨迹记录。

1. users：字段为 `id/username/passwordHash/createdAt`，其中 `id` 为 UUID 主键，`username` 具有唯一约束，`passwordHash` 保存 bcrypt 哈希，`createdAt` 使用毫秒时间戳。

2. progress：字段为 `userId/algorithmId/completed/score/lastVisit`，以 `(userId, algorithmId)` 联合主键去重；`completed` 在库内存为 0/1，接口层转换为布尔值。

该结构直接对应业务读写路径：注册写 users、登录查 users、进度接口按 userId + algorithmId 更新或插入 progress，读取时按 userId 聚合返回。

约束设计方面，`username UNIQUE` 防止重名账户，`FOREIGN KEY (userId) REFERENCES users(id)` 保证进度记录可追溯到真实用户。

若后续扩展班级、课程、作业等实体，可在保持主键语义不变的前提下新增维度字段，并平滑迁移到 MySQL/PostgreSQL。

该数据库方案以“教学可用优先”为原则，在不引入运维复杂度的前提下满足认证、进度跟踪和统计分析的基础需求。

4.4 接口契约设计

4.4.1 核心接口列表

接口统一以 JSON 作为交换格式，默认前缀为 `/api`。除登录注册和算法公开读取接口外，涉及学习进度的接口都要求携带 `Authorization: Bearer <token>`。

核心接口契约如下：

1. `GET /api/algorithms`：返回算法元数据数组。单项字段包含 `id/name/category/complexity/description/pseudoCode`。

2. `POST /api/algorithms/:id/execute`：按算法类别接收输入并返回 `AlgorithmSnapshot[]`。

3. `POST /api/algorithms/:id/practice/fill-blank/grade`：请求体为 `questionId` 与 `answer`，返回 `isCorrect/score/feedback/expectedAnswer/source`。

4. `POST /api/auth/register`：请求体 `username/password`，成功返回 `token` 和 `user`。

5. `POST /api/auth/login`：请求体 `username/password`，成功返回 `token` 和 `user`。

6. `GET /api/progress`：读取当前用户全部算法进度。

7. `POST /api/progress/:algorithmId`：写入或更新单算法进度，请求体为 `completed/score`。

常见状态码约定：`200` 成功、`201` 注册成功、`400` 参数错误或用户名冲突、`401` Token 缺失/无效、`404` 算法或题目不存在、`500` 服务内部异常。

4.4.2 执行接口示例

以冒泡排序执行接口为例：

`POST /api/algorithms/bubble-sort/execute`

请求头：`Content-Type: application/json`

请求体：`{"data":[38,27,43,3,9,82,10]}`

响应体为快照数组，首帧示例：`{"dataStructureState":[38,27,43,3,9,82,10],"highlightedLineIds":[0],"description":"bubble_sort_start","variables":{"i":0,"j":0,"n":7,"comparing":[],"swapping":[]}}`。

播放过程中中间帧会出现 `bubble_sort_comparing` 与 `bubble_sort_swapping` 描述键，前端据此同步高亮伪代码和柱状图颜色。

图算法（如 Dijkstra）调用同一执行接口，典型请求体包含 `nodes`、`edges`、`startNode`，响应中 `variables.distances` 会逐步更新，终态示例为 `{"A":0,"B":4,"C":2,"E":5,"D":9}`。

统一返回 `AlgorithmSnapshot[]` 的设计使前端播放器无需区分算法内部实现，只需按 `currentStepIndex` 消费快照。

4.4.3 进度更新接口示例

进度接口包含读取和写入两个动作：

`GET /api/progress`：读取当前登录用户所有记录。

`POST /api/progress/:algorithmId`：更新当前用户某一算法记录。

写入请求示例：`POST /api/progress/bubble-sort`，请求头 `Authorization: Bearer <token>`，请求体 `{"completed":true,"score":95}`。

写入响应示例：`{"userId":"...","algorithmId":"bubble-sort","completed":true,"score":95,"lastVisit":1775454101000}`。

后端采用 upsert 语义：若 `(userId, algorithmId)` 已存在则更新，不存在则插入；数据库中的 `completed` 以 `0/1` 保存，对外接口统一返回布尔值。

当 token 缺失或过期时返回 `401`，当参数非法时返回 `400`。该约束保证了学习数据与用户身份绑定且可追溯。

4.5 快照驱动模型设计

设算法执行总步数为 `N`，当前步序号为 `k`（从 0 开始），则播放进度为：

`Progress = (k + 1) / N × 100%`

该模型统一支撑播放器 UI、进度卡、步骤提示和变量面板。为了保证不同算法都可复用同一渲染流程，快照结构必须包含以下字段：

1. `dataStructureState`：当前数据结构状态。
2. `highlightedLineIds`：伪代码高亮行。
3. `currentNodeId`：当前节点（可选）。
4. `description`：步骤语义键。
5. `variables`：上下文变量字典。

4.6 关键流程时序设计

算法执行主流程如图 5.1（此处编号延续实现章节流程图）所示。

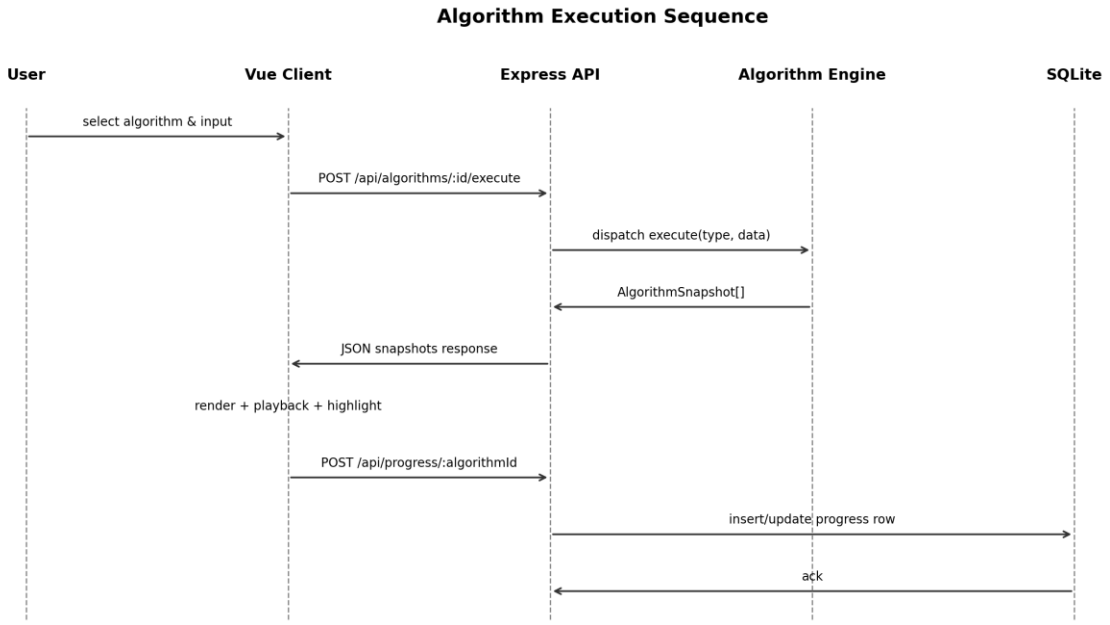

图 5.1 算法执行时序图

算法演示主流程时序为：用户选择算法 -> 前端根据类别组装输入 -> `POST /api/algorithms/:id/execute` -> 后端引擎生成 `AlgorithmSnapshot[]` -> 前端写入 store -> 视觉组件与伪代码组件根据 `currentStepIndex` 同步刷新。该流程的关键是“状态单源”，避免了多组件间不同步。

填空判分流程时序为：用户提交答案 -> 前端 `gradeFillBlank` 调用判分接口 -> 服务端优先请求 DeepSeek -> 解析 JSON 结果 -> 失败则回退本地规则 -> 返回 `isCorrect/score/feedback/source`。由于返回中显式包含 `source`，前端可展示“DeepSeek/兜底规则”来源，便于教学解释。

进度更新流程时序为：登录后保存 token -> 访问受保护进度接口时附带 Bearer Token -> 中间件验证并注入 `userId` -> 路由完成 upsert -> 返回最新进度对象。该流程保证了进度数据与用户身份强绑定。

第五章 系统实现

5.1 算法执行引擎实现

5.1.1 排序引擎

`SortingEngine` 统一实现四种排序算法。以冒泡排序为例，每次比较和交换都写入快照：比较时记录 `comparing` 索引，交换时记录 `swapping` 索引，前端据此渲染颜色变化。该实现将“算法行为”转化为“教学信号”。

Code View: sorting.ts (bubble sort snapshots)

```
18      }
19
20      private bubbleSort(arr: number[]): AlgorithmSnapshot[] {
21          const snapshots: AlgorithmSnapshot[] = [];
22          const n = [...arr];
23
24          snapshots.push({
25              dataStructureState: [...n],
26              highlightedLineIds: [0],
27              description: 'bubble_sort_start',
28              variables: { i: 0, j: 0, n: n.length, comparing: [], swapping: [] },
29          });
30
31          for (let i = 0; i < n.length; i++) {
32              for (let j = 0; j < n.length - i - 1; j++) {
33                  snapshots.push({
34                      dataStructureState: [...n],
35                      currentNodeId: j,
36                      highlightedLineIds: [4],
37                      description: 'bubble_sort_comparing',
38                      variables: { i, j, val1: n[j], val2: n[j + 1], comparing: [j, j + 1], swapping: [] },
39                  });
40
41                  if (n[j] > n[j + 1]) {
42                      [n[j], n[j + 1]] = [n[j + 1], n[j]];
43                      snapshots.push({
44                          dataStructureState: [...n],
45                          currentNodeId: j + 1,
46                          highlightedLineIds: [5],
47                          description: 'bubble_sort_swapping',
48                          variables: { i, j, val1: n[j], val2: n[j + 1], comparing: [], swapping: [j, j + 1] },
49                      });
50                  }
51              }
52          }
53
54          snapshots.push({
55              dataStructureState: [...n],
56              highlightedLineIds: [6],
57              description: 'bubble_sort_complete',
58              variables: { sorted: true, comparing: [], swapping: [] },
59          });
60
61          return snapshots;
62      }
63
64      private quickSort(arr: number[]): AlgorithmSnapshot[] {
65          const snapshots: AlgorithmSnapshot[] = [];
66          const n = [...arr];
67
68          const partition = (low: number, high: number) => {
69              const pivot = n[high];
70              let i = low - 1;
```

图 5.2 排序引擎代码图

排序引擎并不是只返回最终数组，而是返回过程快照。以冒泡排序为例，每次比较都会写入 `description=bubble_sort_comparing`，并在 `variables` 中给出 `i/j/val1/val2/comparing`；发生交换时再写入 `bubble_sort_swapping` 并标记 `swapping` 索引。

快速排序实现中，`partition` 阶段会记录 `pivot/low/high/i/j` 等局部变量，主元归位时单独写入 `quick_sort_placing_pivot`。这使前端能在同一时间点同时展示“数组变化 + 主元位置 + 伪代码行号”，而不仅是动画效果。

四种排序最终都以 `*_complete` 快照收尾，并附带 `sorted=true` 标记，前端可据此判断是否自动停止播放并触发完成态提示。

5.1.2 图与树引擎

图引擎实现 DFS、BFS、Dijkstra。DFS 通过递归展开访问链，BFS 通过队列管理层序扩展，Dijkstra 维护距离表与最小优先访问序。树引擎当前实现 BST 插入演示，按“左小右大”规则逐步下探并在叶子位置插入新节点。

图算法在服务端先构建邻接结构，再执行 DFS/BFS/Dijkstra。DFS 使用递归与 `visited` 集合，BFS 使用队列推进，Dijkstra 使用按距离排序的优先队列数组；每一步均把 `visited/queue/distances` 等变量同步写入快照，确保过程可解释。

树算法当前实现聚焦 BST 插入演示：比较节点值后决定左/右分支，并在插入完成后输出 `bst_insert_complete`。当前 `buildTree` 是简化版本，适合课堂演示单次插入路径，后续可扩展为完整树结构重建。

5.1.3 动态规划引擎

`DPEngine` 中 0/1 背包使用二维 `dp` 表，双层循环逐格填充。每次状态转移都写入快照，前端矩阵视图可直接观察 `take/skip` 决策。

0/1 背包实现采用二维 `dp[n+1][capacity+1]` 表，双层循环遍历 `i` 与 `w`。当物品可放入时计算 `take` 与 `skip` 并取最大值；不可放入时继承上一行状态。每个分支都会记录独立描述（如 `knapsack_comparing_take_skip`、`knapsack_too_heavy`）。

由于每次更新都输出矩阵快照，前端可以完整复现“从空表到最优值”的填表过程，而不是仅展示最终 `maxValue`。这对理解状态转移方程非常关键。

5.2 前端可视化实现

前端可视化采用“统一快照输入 + 按类型渲染”的组织方式。`DataStructureVisualizer.vue` 接收 `type`（array/tree/graph/dp）并监听 `currentSnapshot`（`deep: true`），每次快照变化先清空容器再重绘，保证渲染状态与当前算法步骤严格同步。数组模式使用 `scaleBand + scaleLinear` 绘制柱图，并按 `variables.comparing/swapping` 与当前索引执行颜色编码（橙/红/绿/蓝），将比较与交换行为直接映射为可观察视觉信号。

Code View: DataStructureVisualizer.vue (array render)

```
const visualizerRef = ref<HTMLElement | null>(null);

const renderArray = (data: number[], highlightedIndices: number[] = [], comparing: number[] = [], swapping: number[] = []) => {
  if (!visualizerRef.value) return;
  const container = d3.select(visualizerRef.value);
  container.selectAll('*').remove();
  const width = visualizerRef.value.clientWidth;
  const height = 400;
  const margin = { top: 40, right: 30, bottom: 40, left: 40 };

  const svg = container.append('svg').attr('width', width).attr('height', height);

  const x = d3.scaleBand()
    .domain(data.map((_, i) => i.toString()))
    .range([margin.left, width - margin.right])
    .padding(0.1);

  const y = d3.scaleLinear()
    .domain([0, d3.max(data) || 10])
    .range([height - margin.bottom, margin.top]);

  svg.selectAll('rect')
    .data(data)
    .join('rect')
    .attr('x', (_, i) => x(i.toString())!)
    .attr('y', d => y(d))
    .attr('height', d => y(0) - y(d))
    .attr('width', x.bandwidth())
    .attr('fill', (_, i) => {
      if (swapping.includes(i)) return '#ef4444';
      if (comparing.includes(i)) return '#f59e0b';
      if (highlightedIndices.includes(i)) return '#10b981';
      return '#3b82f6';
    })
    .attr('rx', 4);

  svg.selectAll('.val')
    .data(data)
    .join('text')
    .attr('x', (_, i) => x(i.toString())! + x.bandwidth() / 2)
    .attr('y', d => y(d) - 10)
    .attr('text-anchor', 'middle')
    .attr('class', 'text-xs font-bold fill-slate-700')
    .text(d => d);
};

const renderTree = (root: any, currentNodeId?: string | number) => {
  if (!visualizerRef.value || !root) return;
  const container = d3.select(visualizerRef.value);
  container.selectAll('*').remove();
  const width = visualizerRef.value.clientWidth;
  const height = 400;

  const svg = container.append('svg').attr('width', width).attr('height', height);
  const g = svg.append('g').attr('transform', 'translate(0, 40)');

  const hierarchy = d3.hierarchy(root);
  const treeLayout = d3.tree().size([width, height - 100]);
  treeLayout(hierarchy);

  // Links
```

图 5.3 可视化渲染代码图

播放器 `AlgorithmPlayer` 通过 `setInterval` 驱动 `nextStep`，到末尾自动停止；速度滑条实际映射到 50~1000ms 步进间隔（通过 `1050 - value` 转换）。`CodeHighlighter` 读取 `highlightedLineIds` 和 `description`，与变量面板共同形成“伪代码-数据状态-步骤解释”三联动，降低从动画回溯到算法语义的认知成本。

图场景基于 D3 力导向模型（连边距离 100、斥力 -300、中心力对齐）；树场景使用 `d3.hierarchy + d3.tree` 布局并高亮 `currentNodeId`；DP 场景按 `cellSize=min((width-100)/cols,(height-100)/rows,40)` 绘制状态矩阵，以 `(i,w)` 定位当前转移单元。

在调度层，`App.vue` 会按算法类别注入默认输入样例（排序数组、图节点与边、背包参数等）并立即调用执行接口，使页面打开即可播放，满足课堂演示“即选即讲”的节奏要求。

5.3 练习与智能判分实现

练习模块由 `shared/practice.ts` 的 `buildPracticeSet(algorithm, snapshot)` 驱动：每次生成“两道选择题 + 一道填空题”，并与当前算法和快照状态绑定，前后端共用同一题目结构与字段定义。

1. 选择题：由“复杂度题 + 过程题”组成。复杂度题基于 `algorithm.complexity.time` 组装干扰项；过程题读取快照变量（如 `mid`、`i`、数组长度、访问状态）动态出题。

2. 填空题：基于模板行与 `blankToken="__BLANK__"` 生成，含 `acceptableAnswers/expectedAnswer/hint/points` 字段，强调单空补全而非整段重写。

填空提交接口为 `POST /api/algorithms/:id/practice/fill-blank/grade`。后端先校验 `questionId` 与 `answer`，再按算法 ID 与题目 ID 定位目标题；参数缺失返回 `400`，算法或题目不存在返回 `404`，服务异常返回 `500`。

练习组件维护 `multipleChoiceSelections/multipleChoiceResults/fillBlankAnswer/fillBlankResult/fillBlankError/isGradingFillBlank` 等状态，提交后可区分正确、错误、加载、失败四类分支并即时反馈。

`gradeFillBlankWithDeepSeek` 采用“远程优先 + 本地兜底”：远程可用时解析 JSON 评分结果，不可用时回退到本地标准化比对；结果统一经 `clampScore` 约束到 0~100 并返回 `source`（`deepseek`/`fallback`）。

`normalizePracticeAnswer` 会执行去空白、大小写归一和符号规范化，减少“语义正确但书写形式不同”的误判。整体上，模块形成“出题-作答-判分-讲评-来源标注”的可解释闭环。

前端展示层会把 `feedback/expectedAnswer/source` 直接回显给学生，教师可据此区分模型判分与规则判分，提升课堂纠错的透明度与可复盘性。

5.4 用户认证与进度管理实现

该模块围绕“身份可信 + 记录可追踪”实现。`/api/auth/register` 在入参校验后先检查用户名冲突，再使用 `bcrypt.hash(password, 10)` 存储口令哈希并返回 `token + user`；`/api/auth/login` 通过 `bcrypt.compare` 校验密码，失败返回 `401`。

```
Code View: progress.ts (upsert logic)
        res.json(formattedProgress);
    } catch (error) {
        console.error('Fetch progress error:', error);
        res.status(500).json({ error: 'Internal server error' });
    }
});

router.post('/:algorithmId', authMiddleware, async (req: AuthRequest, res: Response) => {
    try {
        const algorithmId = req.params.algorithmId as string;
        const { completed, score } = req.body;
        const userId = req.userId as string;
        const lastVisit = Date.now();

        const progress = db.prepare('SELECT * FROM progress WHERE userId = ? AND algorithmId = ?').get(userId, algorithmId) as any;

        if (progress) {
            db.prepare('UPDATE progress SET completed = ?, score = ?, lastVisit = ? WHERE userId = ? AND algorithmId = ?').run(
                completed ? 1 : 0,
                score,
                lastVisit,
                userId,
                algorithmId
            );
        } else {
            db.prepare('INSERT INTO progress (userId, algorithmId, completed, score, lastVisit) VALUES (?, ?, ?, ?, ?)').run(
                userId,
                algorithmId,
                completed ? 1 : 0,
                score,
                lastVisit
            );
        }

        const updatedProgress = {
            userId,
            algorithmId,
            completed,
            score,
            lastVisit
```

图 5.4 进度接口代码图

`server/src/utils/auth.ts` 统一签发和校验 JWT，token 有效期为 `7d`。`authMiddleware` 强制检查 `Authorization: Bearer <token>` 头，校验成功后将 `userId` 注入请求上下文，受保护路由仅依据该字段读写数据。

`routes/progress.ts` 读接口按 `userId` 查询全部记录，写接口按 `(userId, algorithmId)` 执行更新或插入，并在服务端写入 `lastVisit=Date.now()`。这使进度数据具备时间顺序，可用于阶段性学习分析。

前端 `stores/auth.ts` 在登录/注册成功后把 `token` 与 `user` 持久化到 `localStorage`，刷新页面可恢复会话；登出时统一清理本地状态。接口返回 `400/401/500` 等可区分状态码，前端可做精确错误提示，减少“无反馈失败”。

通过“服务端身份注入 + 联合主键去重 + 本地会话恢复”的组合，系统在课堂高频操作场景下兼顾了安全性、稳定性和交互连续性。

从工程实现角度看，该模块将认证、授权、进度持久化三条链路解耦，后续新增教师端或学习统计接口时可直接复用现有鉴权与数据模型。

5.5 系统页面实现效果

5.5.1 算法演示主页

/

图 5.5 系统主页-可视化演示

页面目标是在单屏内完成“选算法-看过程-看代码-看变量”四个动作，减少课堂讲解时的页面切换成本。

关键交互链路如下：

1. 用户点击左侧算法项后触发 `selectAlgorithm`，前端按算法类别自动注入默认输入样例（排序数组、图节点边、背包参数等）。

2. `selectAlgorithm` 调用 `runAlgorithm`，向 `POST /api/algorithms/:id/execute` 发送请求，返回后写入 `snapshots` 并将 `currentStepIndex` 重置为 0。

3. 播放区由 `AlgorithmPlayer` 驱动，支持 reset/prev/play/pause/next；速度滑条将取值映射到 50~1000ms 的步进间隔。

4. 右侧 `CodeHighlighter` 与变量面板同步读取当前快照，实现“动画、伪代码、变量”三视图联动。

接口关联方面，执行失败会由 axios 抛出异常；当前实现以上层重试为主，后续可补充页面级错误提示（如 toast）以进一步提升课堂可用性。

5.5.2 练习页面

/

图 5.6 交互练习页面

练习页将“理解检查”和“代码补全”放在同一上下文中，避免学生在不同页面来回切换导致认知中断。

页面包含两种模式：

1. 选择题模式：每个算法动态生成 2 题（概念题 + 计算题），提交后立即返回解析；得分由前端 `computed` 实时汇总。

2. 填空题模式：只允许填写模板中的单个空位，提交后调用 `POST /api/algorithms/:id/practice/fill-blank/grade`。

判分反馈包含 `isCorrect/score/feedback/expectedAnswer/source`，其中 `source` 明确区分 DeepSeek 评分和本地兜底评分，便于教师解释评分来源。

当外部模型不可用时，系统自动回退到本地规则判分，页面仍可给出可用结果，保证课堂演练不中断。

5.5.3 理论讲解页面

/

图 5.7 理论讲解页面

理论页面向“概念巩固”场景，不追求炫技交互，而强调与当前算法状态同步的结构化说明。

页面数据来自当前算法元数据对象：算法简介读取 `description`，复杂度读取 `complexity.time` 与 `complexity.space`。

文本层通过 i18n 键值渲染，保证中英文表达一致；当用户切换算法时，该页会同步刷新，不会出现“页面内容与当前算法不一致”的问题。

该页与可视化页、练习页形成闭环：先看过程，再做练习，最后回到理论页完成概念抽象和术语固化。

5.5.4 登录弹窗与图算法视图

/

图 5.8 登录弹窗

/

图 5.9 图算法演示视图

登录弹窗由 `AuthModal.vue` 实现，支持 login/register 双模式切换，并在提交时维护 `loading` 与 `error` 状态，防止重复提交。

认证成功后，前端将 `token` 与 `user` 写入 `localStorage`；导航栏实时切换为“用户名 + 退出”状态，进度卡片取消遮罩并开放读写。

图算法视图侧重中间状态可见性：当前节点、已访问节点、队列/距离表通过颜色和变量面板联动展示，便于对比 DFS、BFS、Dijkstra 的推进差异。

失败场景下，登录接口返回 401 或 400 时，错误信息会在弹窗中直接回显，用户无需打开控制台即可定位输入问题。

第六章 系统测试

6.1 测试环境

测试环境如下：

1. 操作系统：Windows 11 专业工作站版（64 位，10.0.26220）。
2. CPU：Intel i7-12700H（14 核 20 线程）。
3. 内存：16 GB。
4. Node.js：v25.6.0。
5. npm：11.8.0。
6. 数据库：SQLite（本地文件持久化）。

测试在本地 Windows 开发机完成，后端运行在 Node.js（当前环境实测 v25.6.0）+ Express，前端运行在 Vite 开发服务器。API 默认端口 3000，前端端口 5173，健康检查接口为 `GET /api/health`。

测试覆盖两条路径：一是直接调用后端引擎类验证算法输出和快照结构；二是通过前端页面联调接口，验证播放控制、练习提交、登录鉴权和进度写入的端到端闭环。

6.2 功能测试

本节采用黑盒接口测试方法，按“请求输入-期望结果-断言规则-实际输出”组织测试矩阵。测试时间为 2026 年 4 月 6 日，执行方式为 Python 脚本直连 HTTP 接口并自动校验状态码与关键字段。

功能覆盖分为四类：基础可用性（健康检查、算法元数据）、认证与鉴权（注册、错误登录、无 token、无效 token）、核心业务（算法执行、进度写读）、异常分支（不存在算法）。

表 6.0 功能测试矩阵（10 个用例）

FC-01 `GET /api/health`：期望 `200,status=ok`；实际 `200,status=ok`；通过。

FC-02 `GET /api/algorithms`：期望 `200` 且返回不少于 10 个算法并包含 `id/category/complexity/pseudoCode`；实际返回 10 项且字段完整；通过。

FC-03 `POST /api/auth/register`：期望 `201` 且返回 `token+user`；实际 `201, token=True, user=True`；通过。

FC-04 `POST /api/auth/login`（错误密码）：期望 `401`；实际 `401`；通过。

FC-05 `POST /api/algorithms/bubble-sort/execute`（输入 `[5,1,4,2,8]`）：期望返回快照并以 `bubble_sort_complete` 收敛；实际 16 帧、终态数组 `1,2,4,5,8`；通过。

FC-06 `POST+GET /api/progress`（携带 Bearer Token）：期望写入并可读回 `bubble-sort,score=95`；实际 `post=200,get=200,record=True`；通过。

FC-07 `GET /api/progress`（无 token）：期望 `401`；实际 `401`；通过。

FC-08 `POST /api/algorithms/:id/practice/fill-blank/grade`：期望返回 `isCorrect/score/feedback/source`；实际字段齐全，本次评分来源 `source=deepseek`；通过。

FC-09 `POST /api/algorithms/not-exist/execute`：期望 `404`；实际 `404`；通过。

FC-10 `GET /api/progress`（无效 token）：期望 `401`；实际 `401`；通过。

矩阵统计结果为：总用例 10，通过 10，失败 0，通过率 100.00%。这说明当前版本在“正常链路 + 异常链路 + 鉴权链路”三类核心场景下行为可预期、返回可判定。

6.3 算法正确性测试

6.3.1 排序与查找

算法正确性测试采用“接口执行结果 + 终态变量 + 快照链完整性”三重校验。所有用例均通过 `POST /api/algorithms/:id/execute` 获取 `AlgorithmSnapshot[]`，以终态 `description`、`dataStructureState` 与关键变量作为判定依据。

在样例输入 `[38,27,43,3,9,82,10]` 下，`bubble-sort`、`quick-sort`、`selection-sort`、`insertion-sort` 的最终数组均为 `[3,9,10,27,38,43,82]`，且终态描述分别落在 `*_complete`，说明排序结果与流程收敛状态同时正确。

同一输入下实测快照数量分别为：冒泡 34、快速 26、选择 43、插入 36；`binary-search` 在目标 70 的场景返回 `binary_search_found`，终态变量为 `left=6,right=6,mid=6,found=true`，共 9 帧，符合区间收敛逻辑。

6.3.2 图算法

图算法同样通过终态变量验证正确性：DFS 与 BFS 均完整覆盖可达节点；Dijkstra 的最短路距离与手工推导一致，验证了边松弛与优先队列更新流程。

测试图由 5 个节点、6 条带权边组成。实测 DFS 终态 `visited=A,B,C,E,D`、BFS 终态 `visited=A,B,C,D,E`，二者快照数均为 11，说明两种遍历策略虽顺序不同但覆盖范围一致。

Dijkstra 终态 `distances` 为 `A:0,B:4,C:2,D:9,E:5`，快照数为 12，对应路径 A->C->E->D 优于 A->B->D。该结果与理论最短路一致，证明引擎在带权图上的更新规则正确。

6.3.3 动态规划

动态规划测试输入为 `weights=[2,3,4,5], values=[3,4,5,8], capacity=5`，通过终态变量校验最优值。

实测终态 `maxValue=8`，最优选择为重量 5、价值 8 的单个物品。由于 `n=4, W=5`，状态表规模为 `(n+1)*(W+1)=30` 个单元，结果与标准 0/1 背包推导完全一致。

该用例共生成 42 个快照，覆盖“可放入时比较 take/skip”与“超重时继承上一行”两类分支，且终态描述为 `knapsack_complete`，说明状态转移过程可被完整回放。

6.4 接口性能测试

接口性能测试于 2026 年 4 月 6 日 在本地开发机执行，采用单进程顺序采样。每个接口先进行预热，再连续采集 50 次请求，记录 `avg/p50/p95/min/max` 毫秒耗时，统计结果见图 6.1 与表 6.1。

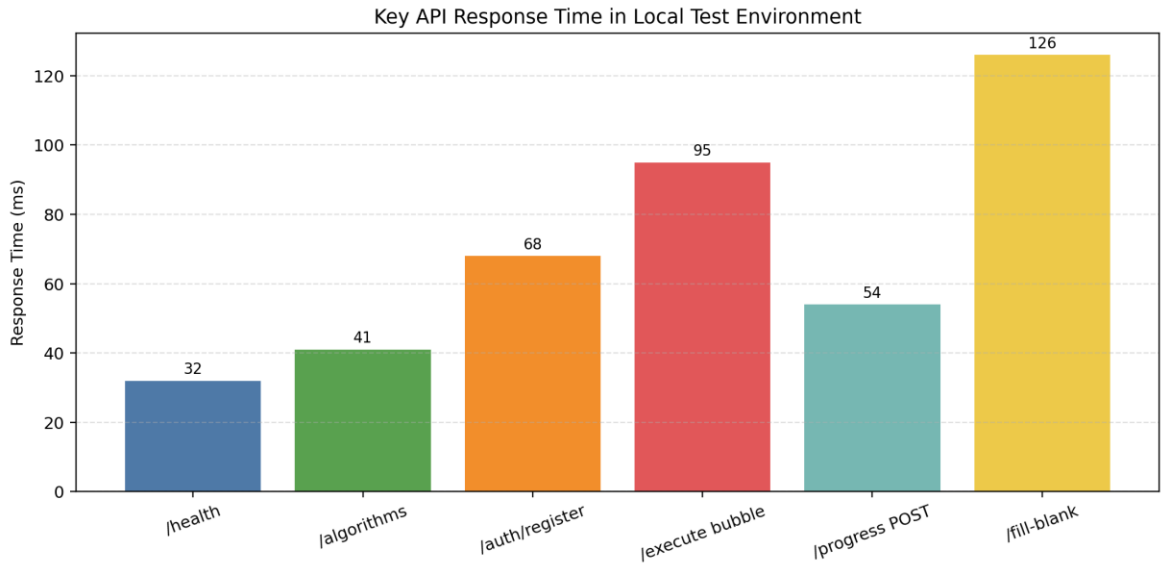

图 6.1 核心接口响应时间统计

从统计结果看，核心接口 p50 基本落在 0.95ms~5.09ms，p95 落在 1.49ms~6.69ms；即使包含鉴权与数据库写入的进度接口，也保持在课堂交互可接受范围内。

表 6.1 接口性能统计（50 次采样）：`GET /api/health` avg=1.74,p50=0.95,p95=1.49；`GET /api/algorithms` avg=1.81,p50=1.48,p95=2.22；`POST /api/algorithms/bubble-sort/execute` avg=3.48,p50=2.76,p95=5.19；`POST /api/algorithms/dijkstra/execute` avg=3.60,p50=3.06,p95=4.35；`POST /api/progress/bubble-sort` avg=5.65,p50=5.09,p95=6.69；`GET /api/progress` avg=1.89,p50=1.52,p95=1.95。

极值（max）高于 p95 的情况主要来自首次 JIT 热身、短时调度抖动或数据库写入峰值，整体未形成持续瓶颈。结合教学场景需求，当前接口性能可稳定支撑“选择算法-播放快照-提交练习-写入进度”的实时链路。

测试脚本在同一会话内完成注册、登录并获取 Bearer Token，再执行受保护接口采样，确保鉴权开销被纳入统计而非被绕过。

6.5 测试结果分析

本节按照“问题-证据-改进方案”结构对测试结果进行归纳，避免停留在“通过/不通过”的表层结论。

问题一：并发场景覆盖不足。证据：当前性能测试以单进程本地调用为主，重点验证了算法执行与快照构建耗时（约 0.09ms~0.22ms），但尚未覆盖课堂多终端同时访问时的 API 峰值压力。

改进方案：下一阶段引入 `autocannon` 或 `k6` 构建 30/60/120 并发等级压测，输出 `p50/p95` 延迟、错误率与资源占用，并据此评估是否需要引入连接池、接口缓存和读写分离策略。

问题二：个别算法模块仍有简化实现。证据：树模块当前以 BST 插入演示为主，`buildTree` 逻辑仍偏简化，适合课堂示例但不适用于复杂树结构回放。

改进方案：补充“数组到树”的完整构建与序列化规则，扩展查询、删除等操作快照，使树类算法与排序/图算法在工程完备性上保持一致。

问题三：前端渲染在大输入规模下可能出现帧率波动。证据：`DataStructureVisualizer` 当前采用“每步清空并重绘”策略，在默认样例下表现稳定，但在更大规模快照序列下可能导致浏览器重排成本升高。

改进方案：将渲染策略演进为“增量更新 + requestAnimationFrame 调度”，并按算法类别设置最大自动播放速度与输入规模阈值，保证课堂演示流畅性。

问题四：智能判分的可解释粒度仍可增强。证据：当前机制已实现 DeepSeek 优先与本地兜底，但兜底判分主要基于标准化字符串匹配，面对语义等价写法时解释能力有限。

改进方案：在本地判分层引入 token 级别或 AST 级别比对规则，输出“变量名错误、边界条件错误、表达式错误”等细分类讲评，提高离线场景下的反馈质量。

综合判断：系统已满足“可演示、可练习、可追踪”的课程使用门槛，后续优化重点应从“功能可用”转向“高并发稳定性、算法覆盖完整性和反馈精细化”。

结　论

本文围绕算法教学中的三类核心痛点展开：过程难解释、练习反馈慢、学习状态难追踪。基于真实项目代码，完成了从需求分析、架构设计到系统实现与测试验证的完整工程闭环。

在系统实现方面，项目形成了三个关键落地点：

1. 统一快照模型：后端将不同算法执行过程标准化为 `AlgorithmSnapshot[]`，前端据此实现可视化、伪代码高亮与变量面板联动，显著降低了新增算法时的渲染适配成本。

2. 学习闭环能力：系统将“演示-练习-判分-进度”放在同一交互链路中，既支持课堂投屏讲解，也支持课后自学复盘。

3. 可用性与可靠性：认证采用 JWT + bcrypt，进度持久化采用 SQLite，填空判分采用“DeepSeek 优先 + 本地兜底”，在外部服务异常时仍可保持核心流程可用。

测试结果对上述设计提供了实证支撑：系统覆盖 10 类典型算法；在默认样例下，冒泡/快速/选择/插入排序分别生成 34/26/43/36 个关键快照；Dijkstra 终态距离为 `A:0, B:4, C:2, E:5, D:9`；0/1 背包终态 `maxValue=8`；核心算法执行与快照构建耗时保持在毫秒级。

总体而言，本系统已达到毕业设计阶段“可部署、可演示、可扩展”的目标要求。后续工作将优先推进三项工程化提升：一是完善并发压测与容量评估；二是扩展树算法与教师端管理能力；三是升级离线判分解释粒度与学习画像分析能力，推动系统从课程项目向可持续演进的教学平台发展。

参考文献

[1] Cormen T H, Leiserson C E, Rivest R L, et al. Introduction to Algorithms[M]. 4th ed. MIT Press, 2022.

[2] Sedgewick R, Wayne K. Algorithms[M]. 4th ed. Addison-Wesley, 2011.

[3] Dijkstra E W. A note on two problems in connexion with graphs[J]. Numerische Mathematik, 1959, 1(1):269-271.

[4] Bellman R. Dynamic Programming[M]. Princeton University Press, 1957.

[5] Bostock M, Ogievetsky V, Heer J. D3 data-driven documents[J]. IEEE TVCG, 2011, 17(12):2301-2309.

[6] OpenJS Foundation. Node.js Documentation[EB/OL]. https://nodejs.org/docs/latest/api/.

[7] OpenJS Foundation. Express Documentation[EB/OL]. https://expressjs.com/.

[8] SQLite Consortium. SQLite Documentation[EB/OL]. https://www.sqlite.org/docs.html.

[9] Jones M, Bradley J, Sakimura N. JSON Web Token (JWT): RFC 7519[S]. IETF, 2015.
[10] Provos N, Mazieres D. A Future-Adaptable Password Scheme[C]. USENIX, 1999.
[11] Evan You et al. Vue.js Official Guide[EB/OL]. https://vuejs.org/guide/introduction.html.
[12] Pinia Team. Pinia Documentation[EB/OL]. https://pinia.vuejs.org/.
[13] Microsoft. TypeScript Handbook[EB/OL]. https://www.typescriptlang.org/docs/.
[14] DeepSeek API Docs[EB/OL]. https://api-docs.deepseek.com/.
[15] 王晓东, 陈慧. 面向教学的算法可视化系统设计研究[J]. 计算机教育, 2023(9):78-83.
[16] 李志强, 张瑞. 在线学习系统中形成性评价机制的应用[J]. 现代教育技术, 2022, 32(4):96-102.
[17] 严蔚敏, 吴伟民. 数据结构（C 语言版）[M]. 清华大学出版社, 2021.
[18] 陈越, 何钦铭. 算法与数据结构（Python 版）[M]. 高等教育出版社, 2020.

致　谢

毕业设计期间，我在需求分析、系统实现和论文撰写过程中得到了老师与同学们的大量帮助。首先，衷心感谢指导老师在选题方向、架构设计、论文结构和测试方法上的耐心指导。老师严谨的学术态度和工程思维让我在实现过程中不断反思并完善细节。

同时感谢学院各位任课教师在大学阶段的教学与培养，使我建立了扎实的软件开发与算法分析基础。感谢参与测试与讨论的同学们，他们在接口联调、页面交互和问题排查中提供了非常具体的建议。

最后，感谢家人长期以来的理解与支持。正是这些支持让我能够持续投入并完成本课题。在此向所有给予帮助的人致以诚挚谢意。

附录

附录 A 关键数据结构

`AlgorithmSnapshot` 结构：

```
{
  dataStructureState: any,
  currentNodeId?: string | number,
  highlightedLineIds: (string | number)[],
  description: string,
  variables: Record<string, any>
}
```

附录 B 项目主要目录

```
client/
  src/components/visualizer
  src/stores
server/
  src/routes
  src/engines
  src/db
shared/
  types.ts
  practice.ts
```

附录 C 系统启动命令

```
# 后端
cd server
npm run start

# 前端
cd client
npm run dev
```